Journal of Visual Communication and Image Representation 50 (2018) 123–134

Contents lists available at ScienceDirect

Journal of Visual Communication and Image Representation

journal homepage: www.elsevier.com/locate/jvci

Shape decomposition-based handwritten compound character recognition for Bangla OCR

Rahul Pramanik*, Soumen Bag

Dept. of Computer Science & Engineering, Indian Institute of Technology (ISM) Dhanbad, Dhanbad 826004, India

ARTICLE INFO

Keywords:
Bangla OCR
Chain code histogram
Handwritten compound character
MLP
Segmentation
Shape decomposition

ABSTRACT

Proper recognition of complex-shaped handwritten compound characters is still a big challenge for Bangla OCR systems. In this paper, we propose a novel shape decomposition-based segmentation technique to decompose the compound characters into prominent shape components. This shape decomposition reduces the classification complexity in terms of less number of classes to recognize, and at the same time improves the recognition accuracy. The decomposition is done at the segmentation area where the two basic shapes are joined to form a compound character. We use chain code histogram feature set with multi-layer perceptron (MLP) based classifier with backpropagation learning for classification. On experimentation, the proposed method is observed to provide good recognition accuracy comparing with other existing methods.

1. Introduction

Recognition of offline handwritten characters is an ongoing topic of research. Several work has been published on recognition of Roman [1,2], Arabic [3–5], Chinese [6–8], and Japanese [9,10] scripts, but only a handful of studies have been done on recognition of printed as well as of handwritten characters in Bangla script [11–16]. The presence of complex-shaped compound (also known as conjunct) characters with their cursive nature in Bangla script makes the recognition problem much more difficult when compared with other scripts [17]. The number of research work further decreases when recognition of compound characters in Bangla script is considered. Garain and Chaudhuri [18] have used feature and run based normalized template matching technique for the recognition of printed Bangla compound characters. Pal et al. [19] have proposed a method to recognize 138 classes of compound characters by using modified quadratic discriminant function (MQDF). Directional information acquired from the arc tangent of the gradient is used as features. Das et al. [20] have introduced a technique that uses multi-layer perceptron (MLP) based classifier and quadtree-based longest run feature to recognize 55 handwritten compound character classes that covers 90% of the total compound character usage. Later, they [21] have extended their work by proposing a MLP and support vector machine (SVM) based strategy that uses shadow and quadtree-based longest run features to recognize 93 character classes including 50 basic and 43 compound handwritten Bangla characters. Das et al. [22] have developed a combined genetic algorithm (GA) and SVM based multi-stage classification for handwritten Bangla compound characters. In this method, SVM is used to perform the first stage of classification followed by GA that handles the classes which were misclassified by SVM. A group of different feature set such as shadow, octant centroid, quadtree-based longest run, and different topological attributes are used to form the overall feature set for the recognition purpose. Bag et al. [23] have proposed a method that decomposes the compound characters into skeletal segments for the improvement of recognition accuracy. In this method, convex shape primitives are extracted to form the structural feature set and template matching scheme is used to recognize the handwritten Bangla compound characters.

We have noticed that a very limited number of work on recognition of complex-shaped Bangla compound characters exist in the literature till now. Now-a-days, researchers are focussing on structural feature set to handle complex structural shape of a compound character. But the extraction of proper structural features from a complex-shaped compound character is itself a big challenge. If by some means the structural complexity of the compound characters be reduced during the preprocessing stages, then we will get better recognition accuracy for such characters. Our work is based on the premise that classification and recognition of two basic characters instead of a complex-shaped compound character would not only provide better recognition accuracy, but also reduce the classification complexity as the number of class diminishes. This is significant as classes for compound characters are no longer needed to be defined separately. To the best of our knowledge no

This paper has been recommended for acceptance by Zicheng Liu.
* Corresponding author.
E-mail addresses: rahul.wbsu@gmail.com (R. Pramanik), bagsoumen@gmail.com (S. Bag).

https://doi.org/10.1016/j.jvcir.2017.11.016
Received 5 March 2017; Received in revised form 19 August 2017; Accepted 19 November 2017
Available online 21 November 2017

work has been published on this aspect for Bangla handwritten complex-shaped compound characters. In this paper, we propose a shape decomposition-based strategy to segment the compound characters into basic shapes, that provides better classification and recognition accuracy and at the same time reduces classification complexity.

The remaining paper is organized as follows. We discuss the characteristics of the Bangla language in Section 2, followed by our proposed methodology in Section 3. The experimental results are discussed in Section 4. Finally, a brief conclusion is presented in Section 5.

2. Characteristics of Bangla language

The Constitution of India registers 22 languages, namely Assamese, Bangla, Bodo, Dogri, Gujrati, Hindi, Kannada, Kashmiri, Konkani, Maithili, Malayalam, Manipuri, Marathi, Nepali, Oriya, Panjabi, Sanskrit, Santhali, Sindhi, Tamil, Telugu, and Urdu. These languages are written in 13 different scripts with over 720 dialects. Bangla script includes Bangla, Assamese, and Manipuri language. The importance of Bangla language can be acknowledged from the fact that it is the seventh most spoken language in the world and the second most spoken language in India [24]. Bangla is also the national language of Bangladesh. Over 83 million people speaks Bangla language comprising nearly 9% of the entire population of India. So, Bangla has a considerable significance as a language and script both. The Bangla script has the following structural and syntactic properties as mentioned below:

- The writing style is from left to right and the concept of upper and lower case is absent in this script.
- Most of the characters in this script have a horizontal line at the upper part of the character called *mātrā* (Fig. 1a).
- The character set of Bangla is divided into two categories: basic and compound characters.
- The basic characters are an agglomeration of vowels and consonants. There are 11 vowels and 39 consonants (Fig. 1b) in this script.
- A compound character is formed by two or more basic characters (Fig. 1c). There are more than 150 compound characters in Bangla. In the present work, we consider 128 compound characters, as we have observed they comprise nearly 95% of the compound characters that appear in a standard piece of Bangla text.

Recently, we did a survey on a very popular Bangla newspaper Anandabazar Patrika (having a circulation of 1.1 million copies) [25] and found that nearly 5% of the entire text comprises of compound characters. This statistic itself emphasizes on how significant compound characters are in Bangla language.

3. Proposed methodology

We propose a shape decomposition-based strategy to segment the compound characters into prominent basic shapes for better classification and recognition accuracy. The proposed methodology is divided into four parts: preprocessing and segmentation area detection, group formation and shape decomposition, feature extraction, and classification. We perform sequential steps to identify the segmentation area where the two basic shapes are amalgamated to form a compound character. Once the area is identified, we examine the presence of a reference line, denoted as *RL*. Based on our knowledge of Bangla script, we define certain rules on the basis of the existence of *RL* to divide the compound characters into five groups. Employing these rules, our proposed algorithm places the compound character in its respective group and decomposes the character into prominent basic shapes. A complete flow diagram of the entire proposed approach is shown in Fig. 2. There exists few compound characters whose structure is much simpler and similar to basic characters, e.g., ঈ, ঊ, ঐ, ঔ, ঊ, etc. Consequently, we have excluded these characters for decomposition, thereby reducing the total number of characters for our experimentation. Fig. 3 provides the whole set of compound characters we consider for our experimentation.

Algorithm 1. Noise Cleaning

Algorithm 1: Noise Cleaning

```
for each ρ_i do
    Size of each connected component ι_j is calculated; ▷ j ∈ [1, p] (p = no. of connected components in ρ_i)
    if ι_j ≤ φ then
        ▷ φ =10 as mentioned in [27]
        κ_i ← ρ_i − ι_j;
    end
    Call Headline_Truncation ();
end
```

3.1. Preprocessing and segmentation area detection

We preprocess each compound character image ψ_i by performing binarization and granular noise cleaning. The binarization algorithm [26] assumes that ψ_i comprises two classes of pixels following bi-modal histogram (foreground and background pixels) and computes the optimum threshold separating the two classes so that their intra-class variance is minimal. The algorithm further utilizes the computed threshold to binarize ψ_i. The binarized image is denoted as ρ_i.

The Noise Cleaning algorithm (1) takes ρ_i as input and calculates the size of each connected component (ι_j). If $\iota_j \leqslant \varphi$ (where, φ is a threshold and its value is held as 10 [27]), the connected component is treated as granular noise and eventually removed.

Algorithm 2. Headline Truncation

Algorithm 2: Headline Truncation

Input : κ_i having dimension m × n with total no. of foreground pixels z.

Output: κ_i with truncated headline and total no. of foreground pixels z', where $z' \leq z$.

```
m_dim ← ⌊m/4⌋; flag_1 ← 0; flag_2 ← 0;
for t ∈ {1, ..., n} do
    for s ∈ {m_dim, ..., m} do
        if κ_i(s, t)=1 & flag_1=0 then
            wdth1_m ← s;
            flag_1 ← 1;
            break;
        end
    end
    break;
end
for t ∈ {n, ..., 1} do
    for s ∈ {m_dim, ..., m} do
        if κ_i(s, t)=1 & flag_2=0 then
            wdth2_m ← s;
            wdth2_n ← t;
            flag_2 ← 1;
            break;
        end
    end
    break;
end
actual_width ← wdth2_m - wdth1_m;
for t ∈ {n, ..., wdth2_n} do
    for s ∈ {1, ..., x} do
        if κ_i(s, t)=1 then
            κ_i(s, t)←0;
        end
    end
end
```

There exist few characters that have elongated headlines (*mātrā*) above them (Fig. 4a). Elongated headlines increase the width (W_d in Fig. 4a) of the entire character which affects in wrong grouping of the compound character. We truncate these extra long headlines to the proper width of the character as shown in Fig. 4b using Algorithm 2. This algorithm takes κ_i as input and determines the actual width of a compound character by examining the lower 3/4th part of κ_i as the headline exists on the upper 1/4th part of κ_i. The algorithm scans each column of the lower 3/4th part from the left side vertically and marks the first encountered foreground pixel, denoted as '$wdth1_m$' as shown in Step no. 2–11. In Step no. 12–22, another scan similar to the previous one is performed but from the right side and marks the first encountered foreground pixel, denoted as '$wdth2_m$'. A simple subtraction provides the actual width of the image. The algorithm marks all pixels outside this range as background pixel in the upper 1/4th part of the image as shown in Step no. 24–30.

We perceive that the average width of a standard stroke (ϑ) is 5 (discussed in Section 4.2 in details). The validation of ϑ is provided in Experimental section. If the average stroke width (ς) of κ_i is greater than ϑ, Algorithm 3 performs one iteration skeletonization on κ_i each time until ς (κ_i) is reduced to ϑ. Skeletonization of κ_i below ϑ produce over-thinned junctions. As the junctions are one of the important feature in structure shape analysis, these over-thinned junctions cannot preserve the true structural shape of the character images. To overcome this problem, we perform partial thinning of κ_i using a parallel thinning method [28] (Fig. 4c).

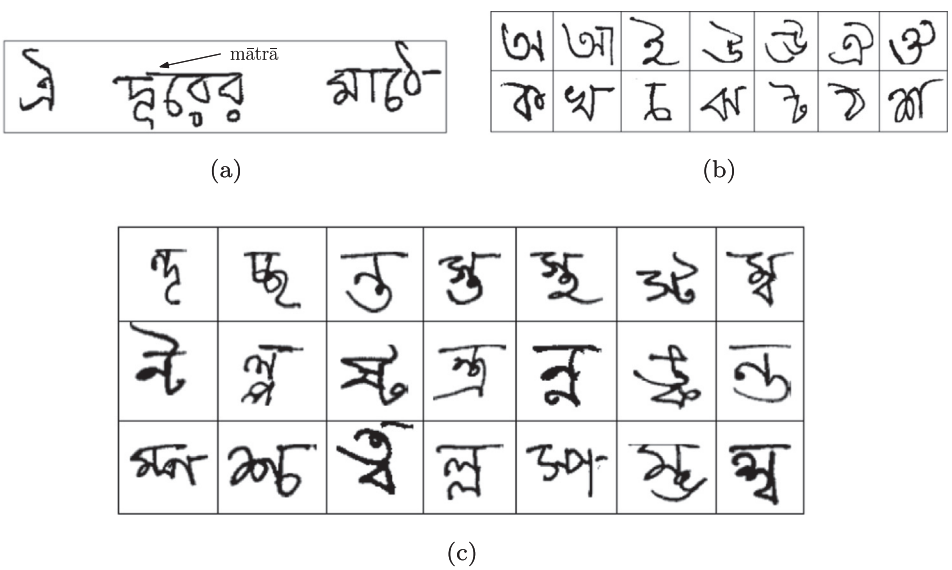

Fig. 1. Structural properties of Bangla script. (a) Horizontal line called *mātrā*; (b) basic characters (top row: vowels, bottom row: consonants); (c) compound characters.

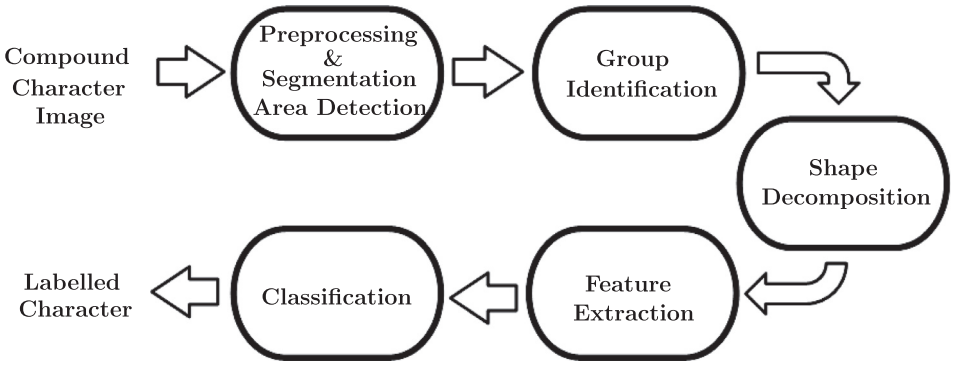

Fig. 2. Flow diagram of the proposed system.

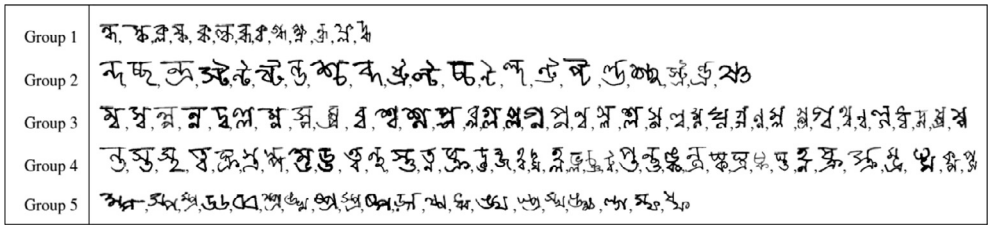

Fig. 3. Complete list of compound characters belonging to each group.

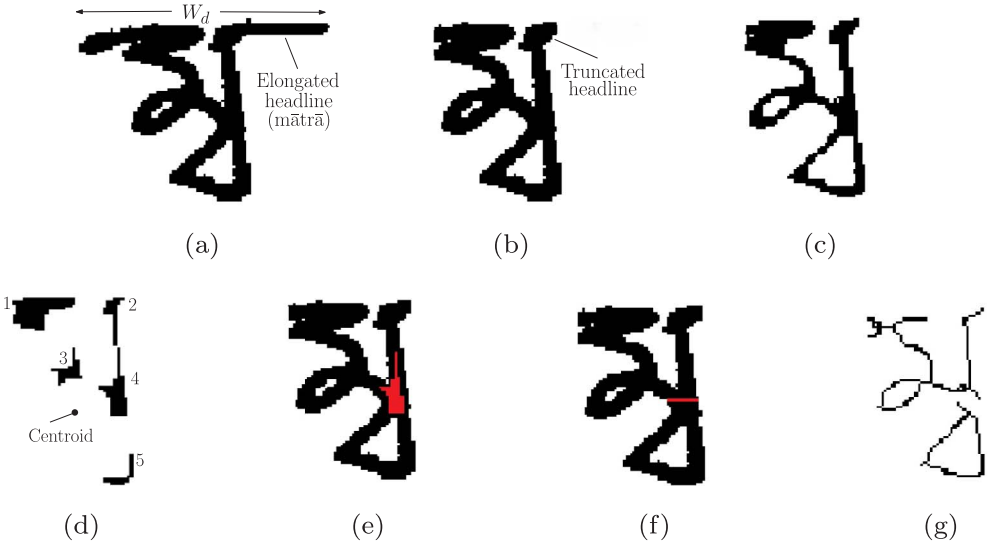

Fig. 4. Stepwise illustration of shape decomposition. (a) After image binarization; (b) after headline truncation; (c) after partial thinning; (d) the closest component (no. 4) among the top 5 components with highest pixel values surrounding the centroid of the whole image; (e) detected segmentation area in the original image marked by red color; (f) after the segmentation where the segmented area is marked with red color; (g) segmented thinned result. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Algorithm 3. Partial Thinning

Algorithm 3: Partial Thinning

```
for each κ_i do
    Calculate ς;
    if ς(κ_i) > ϑ then
        while ς(κ_i) ≠ ϑ do
            Perform one iteration skeletonization on κ_i;        ▷ using [28]
        end
    end
end
```

Algorithm 4 uses a 3 × 3 structural element (Δ) having all one's to erode κ_i which results in splitting κ_i into a number of components (χ_k). The algorithm applies Rosenfeld and Kak component labelling [29] to label all χ_k in the eroded κ_i as $\mathscr{L} = \langle l_1, l_2, \cdots, l_k \rangle$. It sorts $\mathscr{L}$ to get a descending ordered set $\mathscr{L}'$ in terms of pixel values. We infer from our observation that the segmentation area which connects the two consonants would be among the top five components (Fig. 4d) with the highest pixel values. So, the first five components from $\mathscr{L}'$ are stored in τ_l while all the others are discarded. Algorithm 4 calculates the centroid of each τ_l (l ∈ [1,5]) as well as that of κ_i.

Algorithm 4. Component Extraction

Algorithm 4: Component Extraction

```
for each κ_i do
    χ_k ← Apply Δ on κ_i;                    ▷ k ∈ [1, n] (n ← no. of components after erosion of κ_i)
    L ← Apply component labelling on χ_k;    ▷ using [29]
    L' ← Sort L in descending order;
    τ_l ← Extract fisrt 5 components from L';   ▷ l ∈ [1, 5]
    [δ_M^x, δ_M^y] ← Calculate centroid of κ_i;
    for each τ_l do
        [δ_l^x, δ_l^y] ← Calculate centroid of τ_l;
        ξ_l ← Calculate distance between [δ_l^x, δ_l^y] and [δ_M^x, δ_M^y];   ▷ using Equation 1
    end
    ξ' ← Sort ξ in descending order;
    Extract ξ'_1;
end
```

City block distance measurement technique as detailed in Eq. (1) is employed to calculate the distance between the centroid of the image and that of each of the component and stored in ξ_l.

$$dist_i = |\delta_M^x - \delta_l^x| + |\delta_M^y - \delta_l^y| \quad (1)$$

where (δ_M^x, δ_M^y) is the centroid of the whole image and (δ_l^x, δ_l^y), $l \in [1,5]$ is the centroid of the considered component.

The component with minimum distance is the desired segmentation area that connects the two consonants. So, the algorithm sorts ξ to get a descending ordered set ξ'. The closest component (ξ_1') which is considered to be the segmentation area is extracted and its position in κ_i is determined (Fig. 4e). Once we determine the location, a threshold of ±10 is used to find if a reference line (*RL*) (Fig. 5a, b, c) exists on either vertical side of the centroid or not. A validation for choosing this particular threshold value is provided in Section 4.2. If a part of this *RL* does not exist, i.e., we encounter background pixel on one or both end while determining the existence of *RL*, we consider the entire *RL* to be absent in that character (Fig. 5d, e).

If *RL* exists, then we calculate the number of pixels on each side of the line. Number of pixels on left side of *RL* is denoted as NP_L and on the right side of *RL* is denoted as NP_R as shown in Fig. 5a, b.

3.2. Group formation and shape decomposition

We divide the test data set into five groups based on their structural shapes and patterns with respect to their segmentation area as given below:

Group 1 and 2: If *RL* exists and if whole or part of the consonant lies on both side of *RL* and $NP_R \leqslant 20\%$ of NP_L, character is categorized in Group 1 (Fig. 5a). Group 2 is mirror image of Group 1 and differs in last condition, i.e., $NP_R > 20\%$ of NP_L (Fig. 5b).

Group 3: If both consonants lie on the same side of the *RL* (Fig. 5c).

Group 4 and 5: If *RL* (or a part of *RL*) does not exist and $W_h > W_d$, character is categorized in Group 4 (Fig. 5d). Group 5 is mirror image of Group 4 and differs in last condition, i.e., $W_h < W_d$ (Fig. 5e).

Employing these set of rules, our proposed algorithm automatically places the compound character in its respective group. The overall grouping of the characters is shown as a decision tree in Fig. 6. A demonstration of identifying the group of (স্ক) after its segmentation area is detected is shown in Fig. 7. After the grouping, the proposed method performs the following steps.

1. Once the algorithm detects to which of these five groups does each of the compound character belongs, we perform horizontal cut (for Group 1, 3, and 4) and vertical cut (for Group 2 and 5) for the proper decomposition of compound characters. In Fig. 4f, we perform horizontal cut as the given input belongs to Group 3.
2. There exists few compound characters that can be written in two different ways. Let us take the character (ক্ষ) into consideration. When this character is written side-by-side (ক্ষিন), our proposed method detects the character to be in Group 5 and segments it vertically. When the same character is written in a top-down approach (ক্ষ), the algorithm detects it to be in Group 3 and segments it horizontally. So, our method is both flexible and capable of handling the same characters according to their different structural shape.

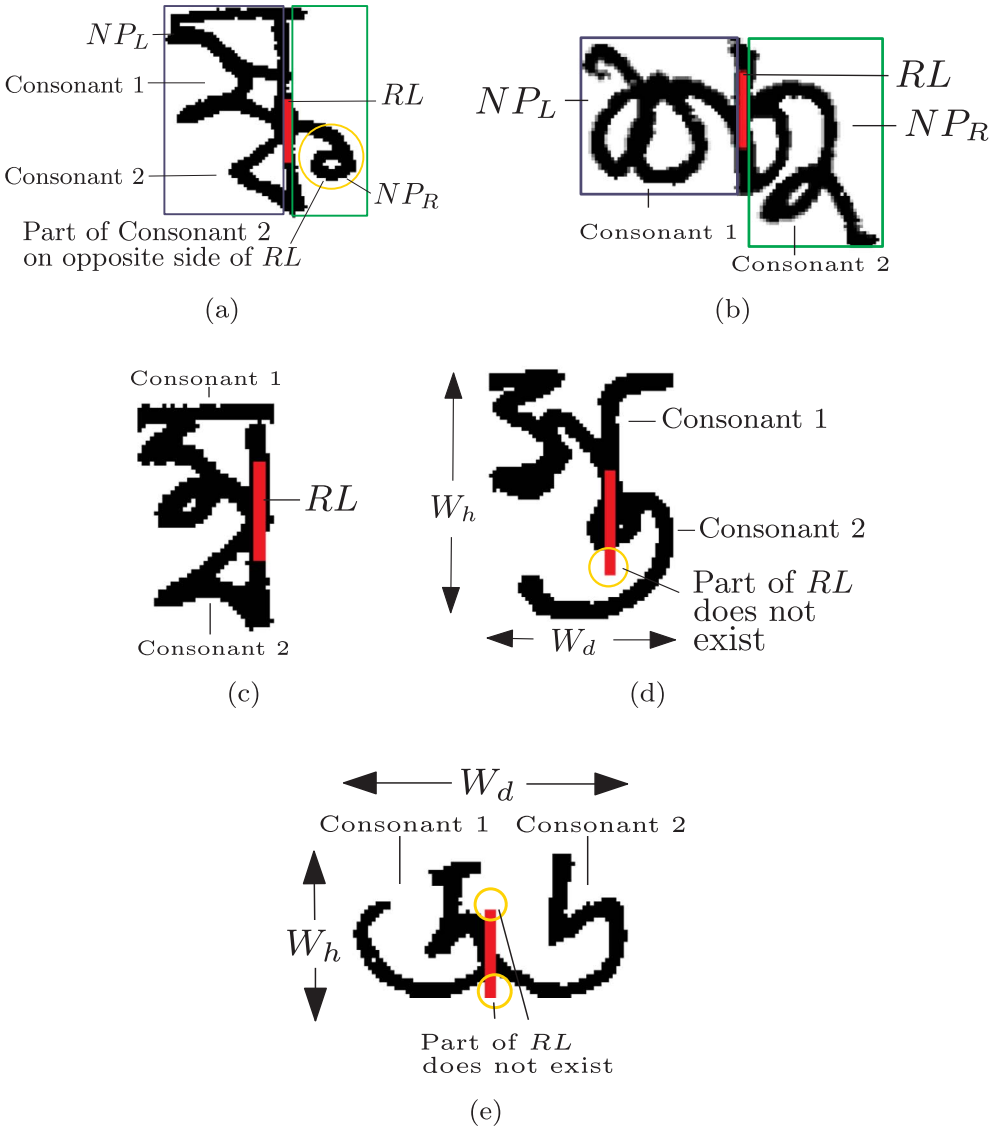

Fig. 5. Group formation: (a) Group 1; (b) Group 2; (c) Group 3; (d) Group 4; (e) Group 5.

Such characters and their corresponding groups are presented in Table 1.

3. There are few pair of compound characters in the current working subset of compound characters that may seem similar when compared visually even though they are placed in different groups. Our algorithm is able to distinguish these characters, as the location of overlapping strokes of each character will be different. One example of such pair of characters is (ন্ড, ন্ড). When segmentation

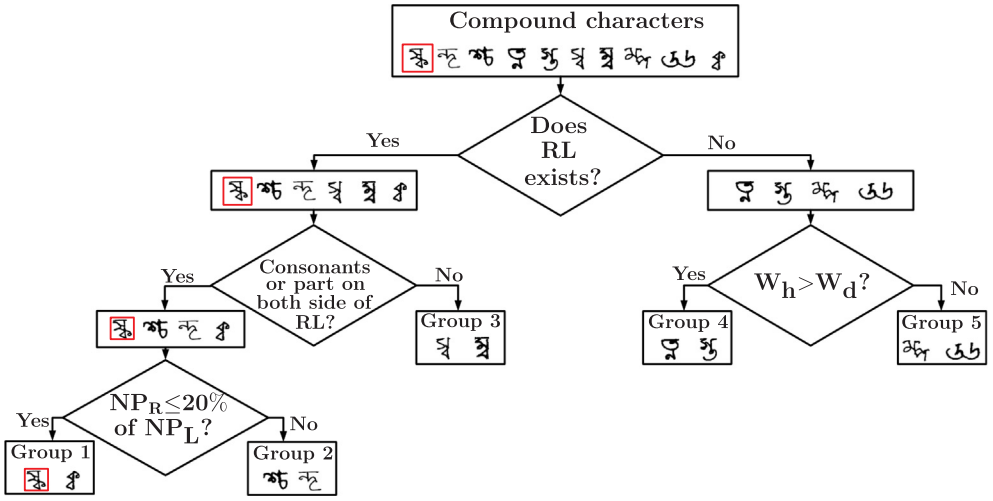

Fig. 6. Decision tree to determine the group of a compound character. As an example the grouping of a compound character is depicted inside red box. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

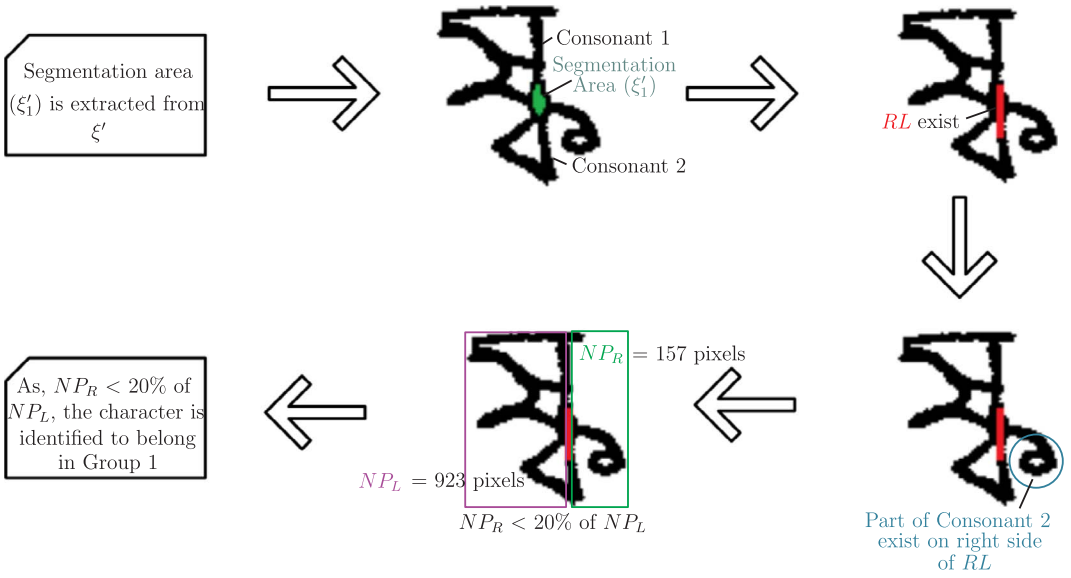

Fig. 7. Demonstration of group identification of (ন্ত্র).

Table 1
Compound characters with more than one form.

Characters with more than one form		Corresponding group	
ব্র	ব্রে	Group 3	Group 5
ক্ষ্ম্র	ক্ষ্ম্র	Group 3	Group 5
ক্ট্রে	ক্ট্রে	Group 4	Group 5
র্ক্ষ্য	র্ক্ষ্য	Group 3	Group 5
র্ম্প্র	র্ম্প্র	Group 3	Group 5

area (ξ_1') is extracted from each of these characters (Fig. 8), the position of ξ_1' differs. ন্ত্র is placed in Group 4 while ন্ত্র is placed in Group 2 based on the analysis performed by our proposed method.

4. We use component labelling algorithm [29] to generate separate images of the basic shapes and apply the same parallel thinning method (as mentioned in Section 3.1) to get the segmented thinned output (Fig. 4g).

3.3. Feature extraction

First, we normalize each image to 49 × 49 pixels. Next, we compute chain code [30] of each image (Fig. 9). We compute the smallest rectangular frame for each of the thinned basic character images and divide each of these images into 7 × 7 equal, non-overlapping blocks. Next, we compute the chain code histogram for each of these blocks.

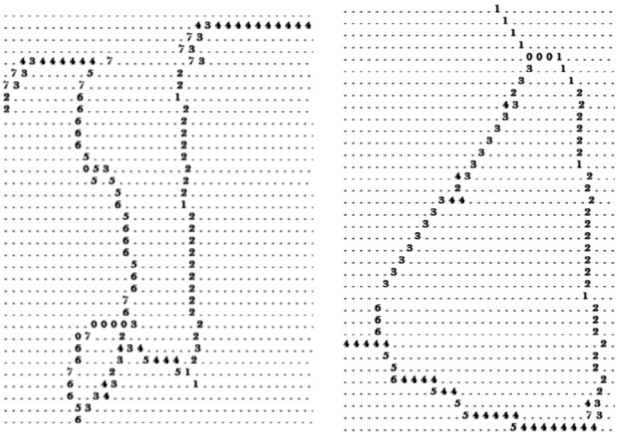

Fig. 9. Characters represented by applying Freeman directional chain code.

Each histogram has one of the 8 possible values, namely 0, 1, 2, 3, 4, 5, 6, and 7. Thus, the feature vector comprises of these local histograms and consists of 7 × 7 × 8 = 392 components.

3.4. Classification

MLP is a feed-forward neural network that maps given input data to expected output data. It constitutes several layers of node, where each layer connects fully to the next. Training corresponds to procuring

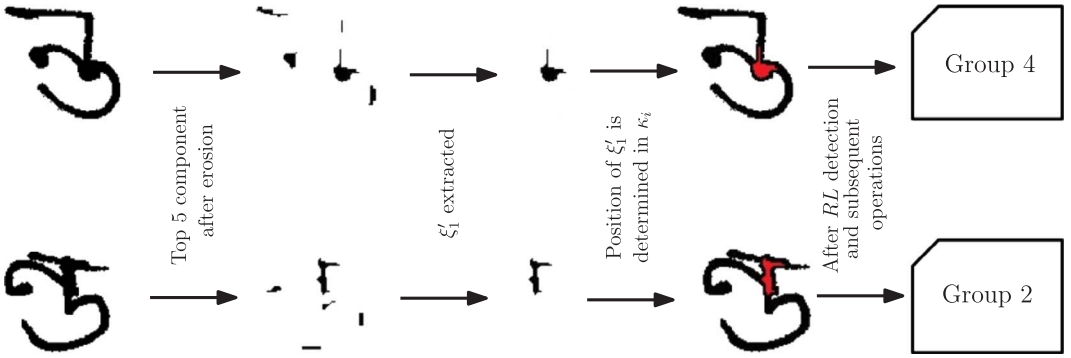

Fig. 8. Visually similar pair of compound characters (ন্ত্র, ন্ত্র) that are correctly distinguished and grouped by our proposed method.

Table 2
Mathematical notations used in the proposed method.

Symbols	Corresponding description	Appeared in
ψ_i	i^{th} Compound character image ▷i = 1, …,8960	Section 3.1
ρ_i	Binarized form of ψ_i	Section 3.1
ι_j	Size of j^{th} connected component ▷j ∈ [1,p]	Section 3.1
φ	Constant (Value set as 10)	Section 3.1
κ_i	Noise cleaned form of ρ_i	Section 3.1
ϑ	Standard average stroke width	Sections 3.1 and 4.2
ς_i	Average stroke width of κ_i	Section 3.1
χ_k	k^{th} component after erosion of κ_i	Section 3.1
Δ	3 × 3 structural element	Section 3.1
$\mathscr{L}$	Labelled connected component set	Section 3.1
$\mathscr{L}'$	Descending ordered $\mathscr{L}$ in terms of number of pixel	Section 3.1
τ_l	l^{th} component of $\mathscr{L}'$ ▷l ∈ [1,5]	Section 3.1
δ_M^x	x-coordinate of the centroid of κ_i	Eq. (1)
δ_M^y	y-coordinate of the centroid of κ_i	Eq. (1)
δ_l^x	x-coordinate of the centroid of l^{th} component	Eq. (1)
δ_l^y	y-coordinate of the centroid of l^{th} component	Eq. (1)
ξ_l	Distance between $[\delta_l^x,\delta_l^y]$ and $[\delta_M^x,\delta_M^y]$	Section 3.1
ξ'	Descending ordered ξ_l	Section 3.1
ξ'_1	First and largest element in ξ'	Section 3.1

suitable weights for all the connections such that an appropriate output is generated for each corresponding input. As a classifier, MLP needs all output nodes to be set to zero except for one node that is marked as one to quadrate the input to the class. The Backpropagation algorithm uses iterative gradient and minimizes the error between the actual and desired outputs of all the nodes in the system for training the MLP. This algorithm learns suitable weights for every pattern in the training data by

- determining the difference between the actual and the desired outputs,
- supplying back this difference (error) level-by-level to the inputs, adjusting the connected weights in a way to modify them in proper proportion to reduce the output difference (error).

In the present work, we use one-hidden layer MLP, so our model consists of an input layer with 392 nodes, a hidden layer with m nodes, and an output layer with 41 nodes. We train the MLP classifier with backpropagation learning. We examine if the basic characters are correctly identified or not. If both the basic characters are correctly identified, we conclude that the compound character formed by these basic characters is correctly recognized.

In the proposed method, we have used several mathematical notations and parameters. A list of these parameters is provided in a tabular form with their corresponding description and section of occurrence in Table 2.

Algorithm 5. Standard Average Stroke Width Detection

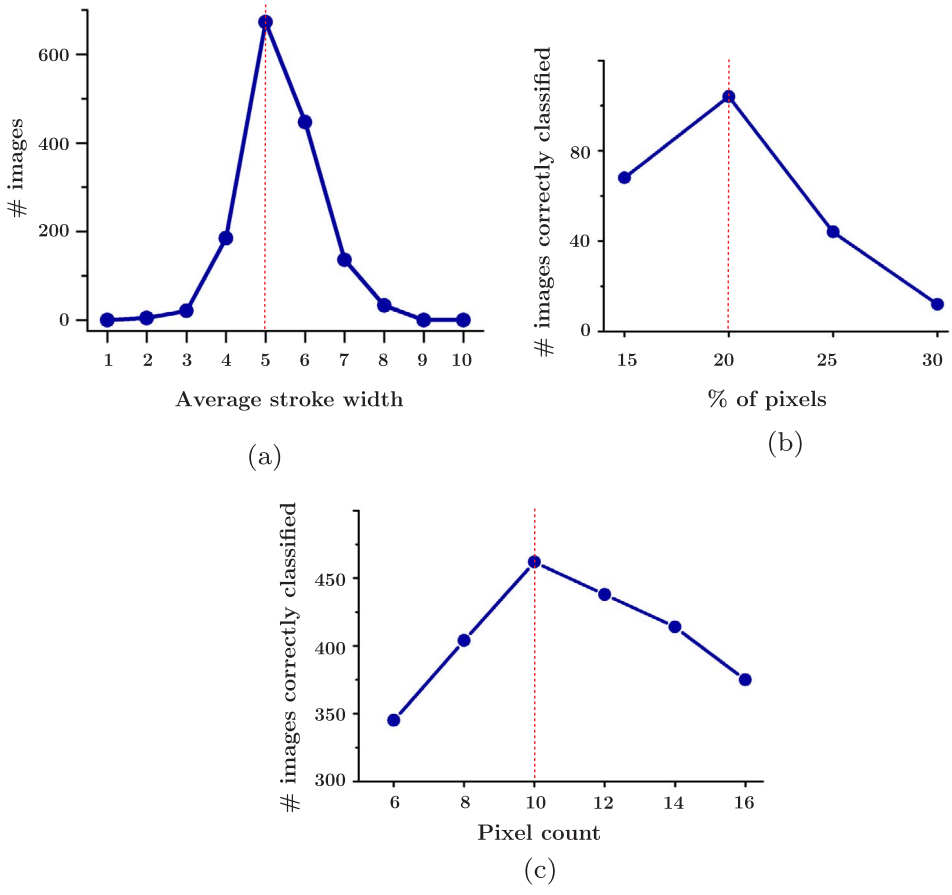

Fig. 10. (a) Validation depicting that 5 is the optimal choice for ϑ; (b) validation depicting that the optimal value of the percentage of pixels for checking whether a character belongs to Group 1 or 2 is 20; (c) validation depicting that taking ±10 pixel as threshold for determination of *RL* provides the best result.

Algorithm 5: Standard Average Stroke Width Detection

Input : 1500 binary images ($img(m,n)$).

Output: Standard average stroke width (ϑ).

```
1  for each img(m,n) do
2      flag ← 0; counter ← 0; max ← 0;
3      for i ∈ {1,...,m} do
4          for j ∈ {1,...,n} do
5              if img(i, j-1)=0 & img(i, j)=1 then
6                  flag← 1; counter←counter+1;
7              end
8              if flag=1 then
9                  if img(i, j-1)=1 & img(i, j)=1 then
10                     counter←counter+1;
11                 end
12                 if img(i, j)=1 & img(i, j+1)=0 then
13                     if counter≥1 & counter≤10 then
14                         wdt[counter]←wdt[counter]+1;
15                     end
16                     flag← 0;
17                     counter← 0;
18                 end
19             end
20         end
21     end
22     for i ∈ {1,...,10} do
23         if wdt[i]≥max then
24             max ← wdt[i];
25             stroke ← i;
26         end
27     end
28     stroke_width[stroke]←stroke_width[stroke]+1;
29 end
30 [num,ϑ]←max[stroke_width];
```

4. Experimental results and analysis

4.1. Dataset

We have used the ICDAR 2013 Segmentation Dataset [31] and Cmaterdb [32] dataset (version 3.1.3.1 for compound characters and version 3.1.2 for basic characters) for our experimental purpose. Both these dataset are unbiased with varied elements and are used by most researchers working in this particular domain. A total of 10,240 sample images of compound characters are used to carry out the experiment. We have used 12,300 basic character images for training purpose. All modules used for implementation are written in MATLAB on the Windows 10 platform.

4.2. Parameter tuning

We have randomly chosen 1500 compound character images from Cmaterdb dataset [32] and have used the Standard Average Stroke Width Detection algorithm (5) to determine standard average stroke width (ϑ) (Fig. 10a). As majority of the strokes in Bangla script are near vertical, we aggregate the number of foreground pixels (1–1 transition) between 0–1 and 1–0 transitions. The algorithm scans every row of each image and changes the flag variable whenever a transition from 0–1 is found as shown in Step no. 5–6. For every 1–1 transition the counter variable is incremented until 1–0 transition is encountered (Step no. 9–10). The value of the counter variable is compared with the position of the 1-D array 'wdt' and the value at that position is incremented by 1 (Step no. 13–15). Consequently, the flag variable is reset back to 0. Once an image is completely processed, the position in 'wdt' having the maximum value is stored in stroke variable and similarly, the value of stroke is compared with 1-D array 'stroke_width' and the respective position value is incremented by 1 as shown in Step no. 21–27. When all the images are processed, the position with maximum value is placed in ϑ (Step no. 30). Fig. 10a depicts that 5 is optimal choice for ϑ.

For a compound character whose *RL* exists, and one of its consonant or part of it exists on the right side of *RL*, the only condition that determines whether it belongs to Group 1 or Group 2 is the measurement and comparison of the amount of pixels on each side of *RL*. We have provided a validation in Fig. 10b which depicts that keeping the percentage of pixels at 20 for comparison provides the best result. We have used 240 random images of the characters belonging to Group 1 to perform this validation.

Fig. 10c provides a validation for the number of pixel chosen to determine whether *RL* exists. We have performed the validation on 400 randomly chosen images from the Cmaterdb database. The figure illustrates that taking ±10 pixel as threshold for determination of *RL* provides the best result.

4.3. Experimental analysis

This section presents the experimental results of our proposed work. We have used thinned segmented compound character images to extract the chain code histogram features. Few of the segmented thinned output of our proposed method is shown in Fig. 11. Table 3 gives a detailed analysis of the recognition performance achieved by each group of compound characters. As per the tabulated results, Group 2 and Group 5 provides the most and least precise result with a recognition accuracy of 92.14% and 86.25% respectively. An overall accuracy of 88.74% is achieved. Recognition rate of few characters is provided in Table 4. We have used the same feature set on SVM and Random Forest classifier and the result is tabulated in Table 5. Although the accuracy achieved by all the 3 classifiers are close to each other, yet MLP provides the best result. We have computed the same feature set for undecomposed compound characters and classified them to provide a comparison of the recognition accuracy achieved using our proposed method keeping the undecomposed character recognition as baseline. The comparison is given in Table 6.

To demonstrate the performance of the proposed method on compound character images with complex background, we have made a dataset containing 400 compound character images. An example of compound character image with complex background is shown in Fig. 12a. We have used an averaging filter of size 3×3 to smooth the image (Fig. 12b) in order to obtain slight better result. Later, as usual, we have applied Otsu's thresholding method [26] (Fig. 12c) and noise normalization (Fig. 12d) and the subsequent steps as mentioned in our proposed method (Fig. 12e). We have achieved an accuracy of 80.50% on the noisy dataset.

Fig. 11. Segmented thinned images of Bangla handwritten compound characters. First column: Input images; Second column: After binarization and noise normalization; Third column: Segmentation of the compound characters into basic structural shapes; Fourth column: The corresponding thinned results.

Table 3
Detailed statistics of each group of compound characters.

Group	Subset of compound characters	Total # samples	# samples correctly detected	Accuracy (%)
1	ক্ক,ক্ত,ক্ম,ক্ষ	1040	921	88.56
2	দৃ,দ্ধ,দ্ভ,দ্ট	1680	1548	92.14
3	শ্চ,শ্ব,শ্ম,ন্ন	2960	2568	86.76
4	স্তু,স্থ,স্ফ,স্তু	2960	2664	90.00
5	ব্দ,ভ্দ,ম্ভ,ড্ড	1600	1381	86.25

Table 4
Character wise assessment of recognition accuracy.

Character	# samples correctly recognized	Accuracy	Character	# samples correctly recognized	Accuracy
	78	97.50		76	95.00
	78	97.50		76	95.00
	78	97.50		75	93.75
	77	96.25		74	92.50
	76	95.00		74	92.50

Table 5
Accuracy achieved using different classifiers.

Group	Classifiers		
	MLP	SVM	Random forest
1	88.56	80.46	81.17
2	92.14	90.50	90.50
3	86.76	85.10	85.10
4	90.00	94.50	92.70
5	86.25	81.70	81.40
Overall	88.74	86.45	86.17

Table 6
Accuracy comparison with undecomposed characters as baseline.

Compound characters	Total # samples	Recognition accuracy (%)
Whole (baseline)	10240	68.76
Decomposed		88.74

4.4. Comparative study

We have compared our proposed method with three other works based on handwritten compound character recognition. Pal et al. [19] have used directional gradient features and modified quadratic discriminant function (MQDF) to recognize 138 compound character classes. Das et al. [20] have used Quad tree based longest run feature set with MLP classifier for identifying compound characters. Das et al. [21] have further employed Quad tree based shadow and longest run feature set along with MLP and SVM classifier separately to classify compound characters. All these methods recognize the compound characters directly, considering each character as a separate class. This kind of approach increases the classification complexity due to the presence of large number of classes and thereby reduces the recognition accuracy.

In our proposed method, we have decomposed the 128 compound characters into 41 basic characters first. As, we have used these basic characters for classification, the number of classes is reduced from 128 to 41. Next, we have extracted chain code histogram feature set and performed the classification using MLP with backpropagation learning. Classification of basic instead of compound characters with simple feature set not only yields better recognition accuracy but also reduces the number of classes comparing with other methods. This is rather encouraging as we did not need to train extra classes which reduces the classification complexity. Table 7 epitomizes the preprocessing methods, features and classifiers used in the above mentioned works along with our proposed method.

We have implemented the two methods [20,21] and used the same datasets and test parameters for comparison. A comparison of our achieved result with these two methods is provided in Table 8.

There are certain compound characters that have near similar

Table 7
Summary of preprocessing methods, features and classifiers used in other works on Bangla handwritten compound character recognition with our proposed method.

Method	Preprocessing	Feature set	Classifier
Pal et al. [19]	Binarization, Median filtering, Size normalization	Directional gradient	MQDF
Das et al. [20]	Not mentioned	Quad tree based longest run	MLP
Das et al. [21]	Binarization	Quad tree based shadow, longest run	MLP, SVM
Proposed method	Binarization, Noise cleaning, Headline truncation, Partial thinning	Chain code histogram	MLP

Table 8
Comparison with other works on Bangla handwritten compound character recognition.

Method	# character type	# classes defined	Test set size	Recognition accuracy (%)
Das et al. [20]	128	128	10,240	65.14
Das et al. [21]		128		71.10
Proposed method		41		88.74

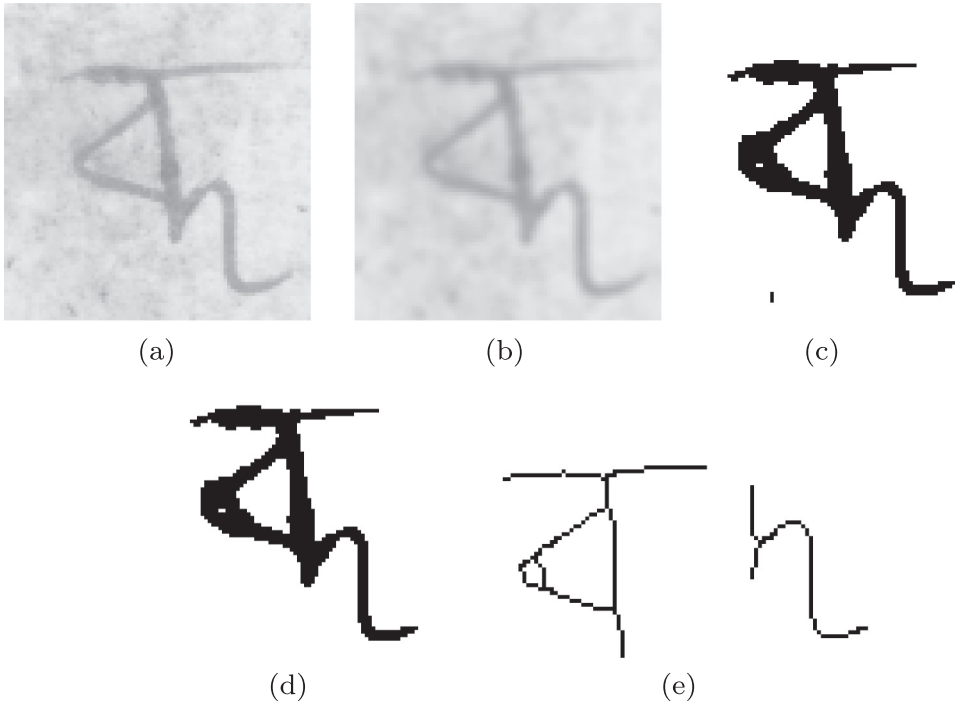

Fig. 12. Preprocessing and shape decomposition of compound character image with complex background. (a) Input image; (b) smoothed using a 3 × 3 averaging filter; (c) after application of Otsu's thresholding; (d) after noise normalization; (e) thinned output after following the subsequent steps of the proposed method.

Table 9
Comparison of rate of confusion for similar compound characters with Pal et al. [19].

Method	Few samples of misclassified characters	% confusion	% confusion in our Proposed Method
Pal et al. [19]		0.13	0.06
		0.09	0.03
		0.11	0.03

structure and the classifier misclassify these characters substantially. Pal et al. [19] have mentioned such characters. As we have segmented the compound character into basic shapes, we have achieved a lesser confusion rate for these characters. Table 9 delineates a comparison based on confusion rate for each of these characters of the proposed method with [19].

5. Conclusion

Recognition of cursive Bangla handwritten characters has always been a challenge for researchers. The presence of compound characters makes the task much more difficult. Reduction of structural complexity of these compound characters would make recognition more accurate. In the current work, we have developed a shape decomposition-based methodology to segment the complex-shaped compound characters into two basic, simple, and prominent shapes. Our achievement is two fold. We not only achieve good recognition rate but at the same time use less no. of classes to achieve such accurate result. Classes for compound characters no longer needs to be defined separately. This work will pave a new path towards exploiting the shape of the compound characters to use their structural features as an effective feature set for Bangla character recognition. In future, we will continue our work to incorporate this methodology in OCR for other Indian languages.

References

[1] H. Bunke, T. Varga, Off-line Roman cursive handwriting recognition, Digital Docum. Process. (2007) 165–183.
[2] F. Li, S. Gao, Character recognition system based on back-propagation neural network, in: International Conference on Machine Vision and Human-Machine Interface (MVHI), 2010, pp. 393–396.
[3] J.H. AlKhateeb, J. Ren, J. Jiang, H. Al-Muhtaseb, Offline handwritten Arabic cursive text recognition using Hidden Markov Models and re-ranking, Pattern Recogn. Lett. 32 (8) (2011) 1081–1088.
[4] M. Rashad, K. Amin, M. Hadhoud, W. Elkilani, Arabic character recognition using statistical and geometric moment features, in: Japan-Egypt Conference on Electronics, Communications and Computers (JEC-ECC), 2012, pp. 68–72.
[5] M.T. Parvez, S.A. Mahmoud, Arabic handwriting recognition using structural and syntactic pattern attributes, Pattern Recogn. 46 (1) (2013) 141–154.
[6] C.L. Liu, F. Yin, D.H. Wang, Q.F. Wang, Online and offline handwritten Chinese character recognition: benchmarking on new databases, Pattern Recogn. 46 (1) (2013) 155–162.
[7] Q.F. Wang, F. Yin, C.L. Liu, Handwritten Chinese text recognition by integrating multiple contexts, IEEE Trans. Pattern Anal. Mach. Intell. 34 (8) (2012) 1469–1481.
[8] Z. Zhong, L. Jin, Z. Xie, High performance offline handwritten Chinese character recognition using GoogLeNet and directional feature maps, in: International Conference on Document Analysis and Recognition (ICDAR), 2015, pp. 846–850.
[9] Y. Sobu, H. Goto, H. Aso, Binary tree-based precision-keeping clustering for very fast Japanese character recognition, in: International Conference of Image and Vision Computing (IVCNZ), 2010, pp. 1–6.
[10] X.D. Zhou, D.H. Wang, F. Tian, C.L. Liu, M. Nakagawa, Handwritten Chinese/Japanese text recognition using semi-Markov conditional random fields, IEEE Trans. Pattern Anal. Mach. Intell. 35 (10) (2013) 2413–2426.
[11] R. Sarkar, N. Das, S. Basu, M. Kundu, M. Nasipuri, D.K. Basu, A two-stage approach for segmentation of handwritten Bangla word images, in: International Conference on Frontiers in Handwriting Recognition (ICFHR), 2008, pp. 403–408.
[12] S. Bag, P. Bhowmick, G. Harit, A. Biswas, Character segmentation of handwritten Bangla text by vertex characterization of isothetic covers, in: National Conference on Computer Vision, Pattern Recognition, Image Processing and Graphics (NCVPRIPG), 2011, pp. 21–24.
[13] S. Mandal, S. Sur, A. Dan, P. Bhowmick, Handwritten Bangla character recognition in machine-printed forms using gradient information and Haar wavelet, in: International Conference on Image Information Processing (ICIIP), 2011, pp. 1–6.
[14] S. Bag, P. Bhowmick, G. Harit, Recognition of Bengali handwritten characters using skeletal convexity and dynamic programming, in: International Conference on Emerging Applications of Information Technology (EAIT), 2011, pp. 265–268.
[15] P.P. Roy, P. Dey, S. Roy, U. Pal, F. Kimura, A novel approach of Bangla handwritten text recognition using HMM, in: International Conference on Frontiers in Handwriting Recognition (ICFHR), 2014, pp. 661–666.
[16] M.M. Rahman, M.A.H. Akhand, S. Islam, P.C. Shill, M.H. Rahman, Bangla handwritten character recognition using convolutional neural network, Int. J. Image, Graph. Signal Process. (IJIGSP) 7 (8) (2015) 42.
[17] S. Bag, G. Harit, A survey on optical character recognition for Bangla and Devanagari scripts, Sadhana 38 (2013) 133–168.
[18] U. Garain, B.B. Chaudhuri, Compound character recognition by run-number-based metric distance, in: Photonics West'98 Electronic Imaging, 1998, pp. 90–97.
[19] U. Pal, T. Wakabayashi, F. Kimura, Handwritten Bangla compound character recognition using gradient feature, in: International Conference on Information Technology (ICIT), 2007, pp. 208–213.
[20] N. Das, S. Basu, R. Sarkar, M. Kundu, M. Nasipuri, D.K. Basu, Handwritten Bangla Compound character recognition: potential challenges and probable solution, in: Indian International Conference on Artificial Intelligence (IICAI), 2009, pp. 1901–1913.
[21] N. Das, B. Das, R. Sarkar, S. Basu, M. Kundu, M. Nasipuri, Handwritten Bangla Basic and Compound Character Recognition using MLP and SVM Classifier, 2010. Available from: < arXiv:1002.4040 > .
[22] N. Das, A. Kallol, R. Sarkar, S. Basu, M. Kundu, M. Nasipuri, A novel GA-SVM based multistage approach for recognition of handwritten Bangla compound characters, in: International Conference on Information Systems Design and Intelligent Applications, 2012, pp. 145–152.
[23] S. Bag, G. Harit, P. Bhowmick, Recognition of Bangla compound characters using structural decomposition, Pattern Recogn. 47 (3) (2014) 1187–1201.
[24] https://en.wikipedia.org/wiki/Bengali_language (online; accessed 05-Jul-2017).
[25] https://en.wikipedia.org/wiki/Anandabazar_Patrika (online; accessed 05-Jul-2017).
[26] N. Otsu, A threshold selection method from gray-level histograms, Automatica 11 (1975) 23–27.
[27] U. Bhattacharya, M. Shridhar, S.K. Parui, P.K. Sen, B.B. Chaudhuri, Offline recognition of handwritten Bangla characters: an efficient two-stage approach, Pattern Anal. Appl. 15 (4) (2012) 445–458.
[28] L. Huang, G. Wan, C. Liu, An improved parallel thinning algorithm, in: International Conference on Document Analysis and Recognition (ICDAR), 2003, p. 780.
[29] A. Rosenfeld, A. Kak, Digital Picture Processing, Academic Press, 1982.
[30] H. Freeman, Computer processing of line-drawing images, ACM Comput. Surv. 6 (1) (1974) 57–97.
[31] N. Stamatopoulos, B. Gatos, G. Louloudis, U. Pal, A. Alaei, ICDAR 2013 handwriting segmentation contest, in: International Conference on Document Analysis and Recognition (ICDAR), 2013, pp. 1402–1406 (online; accessed 10-Jan-2017).
[32] N. Das, K. Acharya, R. Sarkar, S. Basu, M. Kundu, M. Nasipuri, A benchmark image database of isolated Bangla handwritten compound characters, Int. J. Docum. Anal. Recogn. 17 (4) (2014) 413–431. Available: http://archive.is/o/xDqG6/cmaterdb.googlecode.com/files/CMATERdb3.1.3.1.rar (Online; accessed 10-Jan-2017).